

Exam A - School Solution

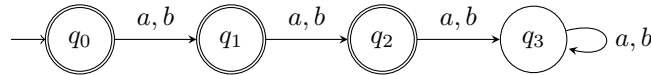
Question 1

The claim is **incorrect**.

L_1 and L_2 have 10 Myhill-Nerode equivalence classes each. We saw that for every language with a finite number k of equivalence classes, there exists a DFA with k states that recognizes it. Hence there exist DFAs A_1 and A_2 with 10 states each that decide L_1 and L_2 respectively. Using the product-DFA construction from class we can therefore construct a DFA with $10 \cdot 10 = 100$ states that decides $L_1 \cap L_2$. Since we know that the number of Myhill-Nerode equivalence classes is bounded above by the size of a DFA that decides a language, we obtain that $L_1 \cap L_2$ has at most 100 Myhill-Nerode classes, proving that the claim is incorrect. $\square$

Question 2

We define the following automaton A that has 4 states.



Notice that the run of A over every word whose length is $l \leq 2$ will end in q_l , and that the runs over words whose length is greater than or equal to 3 end in q_3 . By the definition of all the states as accepting except q_3 we have that $L(A) = \{w \in \{a, b\}^* : |w| \leq 2\}$. The words over $\{a, b\}^*$ whose length is at most 2 are $\{\epsilon, a, b, aa, ab, ba, bb\}$, so $|L(A)| = 7$. $\square$

Question 3

For every $l \in \{0\} \cup \mathbb{N}$, the number of words over Σ whose length is l is 2^l . Therefore, the number of words over Σ whose length is less than k is

$$\sum_{l=0}^{k-1} 2^l = 2^k - 1 < 2^k,$$

where the equality can be derived using the formula for the sum of a geometric series. It is given that $|L(A)| \geq 2^k$, and hence $L(A)$ contains a word whose length is at least k . We saw (in Exercise 2) that if the language of a DFA contains a word whose length is at least its number of states, then its language is infinite. Hence, $L(A)$ is infinite. $\square$

Question 4

The claim is **correct**.

Let A and B be such languages. $A \in \text{R}$ so there exists a TM M that decides A . B is a non-trivial language, so there exist words $w_1 \in B$ and $w_2 \notin B$. We show a reduction from A to B .

Construction: Given an input w , the reduction machine runs M on w , if M accepts - it returns w_1 , and if it rejects it returns w_2 .

Correctness: If $w \in A$, M accepts w , and the reduction machine returns $w_1 \in B$. If $w \notin A$, M rejects w , and the reduction machine returns $w_2 \notin B$.

Computability: M decides A so it halts on every input, and hence also the reduction machine halts on every input.

We have thus shown a reduction from A to B . $\square$

Question 5

We show a reduction from NY-HALT to $\overline{\text{NY-HALT}}$.

Construction: We call an input "valid" if it is of the form of $Y \circ x$ or $N \circ x$, where x is an encoding of a Turing machine and a word. If the input is not valid, the reduction returns $Y \circ \langle M_{\text{HALT}}, \epsilon \rangle$, where M_{HALT} is a machine that immediately halts on every input. If the input is valid, the reduction "flips" its first letter from Y to N or vice versa and leaves the rest unchanged.

Correctness: Note that the word $Y \circ \langle M_{\text{HALT}}, \epsilon \rangle$ is in NY-HALT. Hence, when the input is invalid (and therefore not in NY-HALT), the output of the reduction is in NY-HALT and thus not in $\overline{\text{NY-HALT}}$. Otherwise, the input is valid. Valid inputs are in NY-HALT if and only if the first letter "matches" the halt of the machine on the word. Hence, flipping the first letter transforms words in NY-HALT to words not in NY-HALT, that is, in $\overline{\text{NY-HALT}}$, and vice versa. The reduction flips the first letter and hence the input is in NY-HALT if and only if the reduction's output is in $\overline{\text{NY-HALT}}$.

Computability: Checking if the input is valid and flipping the first letter are both computable. $\square$

Question 6

We will show that $\text{NY-HALT} \in \overline{\text{RE} \cup \text{co-RE}}$.

It suffices to show that $\text{HALT} \leq_m \text{NY-HALT}$ and $\text{NON-HALT} \leq_m \text{NY-HALT}$. Since $\text{HALT} \notin \text{co-RE}$, using the reduction theorem, it will imply that $\text{NY-HALT} \notin \text{co-RE}$. Similarly, since $\text{NON-HALT} \notin \text{RE}$, using the reduction theorem, it will imply that $\text{NY-HALT} \notin \text{RE}$.

1. We show that $\text{HALT} \leq_m \text{NY-HALT}$.

Construction: The reduction is $f(x) = Y \circ x$.

Correctness:

$$f(x) \in \text{NY-HALT} \iff Y \circ x \in \text{NY-HALT} \iff x \in \text{HALT},$$

where the transitions are by the construction and the definition of NY-HALT.

Computability: A Turing machine can add a letter at the beginning of its input.

2. Similarly, $\text{NON-HALT} \leq_m \text{NY-HALT}$ using the reduction $f(x) = N \circ x$ and similar correctness and computability arguments.

We showed both reductions so $\text{NY-HALT} \notin \text{co-RE}$ and $\text{NY-HALT} \notin \text{RE}$, so overall $\text{NY-HALT} \in \overline{\text{RE} \cup \text{co-RE}}$.

Question 7

The claim is **incorrect**.

We assume for contradiction that there exists $k \in \mathbb{N}$ such that $P = \bigcup_{i=1}^k \text{TIME}(n^i)$. For every $i \in [k]$, $\text{TIME}(n^i) \subseteq \text{TIME}(n^k)$, so $P = \bigcup_{i=1}^k \text{TIME}(n^i) \subseteq \text{TIME}(n^k)$.

Notice that $n^k = o\left(\frac{n^{k+1}}{\log(n^{k+1})}\right)$ and n^{k+1} is a time constructible function. Since both conditions are satisfied, we can use the time hierarchy theorem to show that $\text{TIME}(n^k) \subsetneq \text{TIME}(n^{k+1})$.

Overall, we get that $P \subsetneq \text{TIME}(n^{k+1})$ in contradiction to the fact that $\text{TIME}(n^{k+1}) \subseteq P$ (by the definition of P). $\square$

Question 8

First, we show that $\text{NP} \subseteq \text{coNP}$. Let K be a language K in NP. Since L is NP-hard, $K \leq_p L$, and since $L \in \text{coNP}$ by the reduction theorem $K \in \text{coNP}$. Similarly, by choosing an arbitrary language K in coNP and using the coNP-hardness of L and the fact that it is in NP, we can show that $K \in \text{NP}$ and deduce that $\text{coNP} \subseteq \text{NP}$. Overall, we get that the existence of such L implies that $\text{NP} = \text{coNP}$.

Question 9

L is NP-complete.

First, we show that $L \in \text{NP}$ by showing a polynomial verifier that decides it. The verifier takes an encoding of a graph $\langle G \rangle$ and an encoding of a path $\langle P \rangle$ in G . Notice that the length of the encoding of P is polynomial in the length of G . The verifier verifies that the number of vertices in G is even and positive, and that P is a simple path in G whose length is at least $\frac{n}{2} - 1$ (i.e. with at least $\frac{n}{2}$ vertices). All the operations (verifying that a sequence of vertices is a simple path, counting vertices, etc.) can be computed in polynomial time.

Now we show that L is NP-hard. We do it by showing a reduction from HAM – PATH. Since HAM – PATH is NP-hard, by a theorem we saw in class, it will follow that L is NP-hard.

Construction: Given the encoding of a graph G' with n' vertices, the reduction outputs an encoding of a graph G , where G is obtained from G' by adding n' new vertices to it that are all isolated (have no edges). Note that G thus has an even number of $n := 2n'$ vertices.

Correctness: ($\Rightarrow$) If there is a Hamiltonian path P in G' , then P is also a simple path in G of length $n' - 1 = \frac{n}{2} - 1$.

($\Leftarrow$) We assume that G has a simple path P of length $\geq \frac{n}{2} - 1$. If $n' = 1$, G' trivially has a Hamiltonian path with a single vertex. Otherwise, $n' \geq 2$ so $\frac{n}{2} - 1 \geq 1$, and P has at least two vertices (the length of a path is the number of edges it contains, and that equals the number of vertices minus 1). Any simple path in G with more than one vertex uses only vertices from G' , as the vertices we added have no edges. P is therefore a simple path in G' of length $\frac{n}{2} - 1 = n' - 1$, which means that it is a Hamiltonian path in G' , and therefore $\langle G' \rangle \in \text{HAM} - \text{PATH}$.

Time Complexity: Adding n' vertices to the graph can be done in polynomial time.